

Nama : Ahmad Faiz Batubara
NIM : 2411521016
PBD A

STUDI KASUS-1 : SISTEM INFORMASI MANAJEMEN PERPUSTAKAN

Perhatikan table berikut.

1	Latar Belakang	Perpustakaan Universitas Nusantara masih melakukan pencatatan data buku, anggota, dan transaksi peminjaman secara manual. Hal ini menyebabkan beberapa masalah, seperti: • Kesulitan mencari data buku yang tersedia • Pencatatan peminjaman dan pengembalian sering terjadi kesalahan • Laporan peminjaman buku sulit dibuat • Data anggota perpustakaan tidak terorganisir dengan baik Untuk mengatasi masalah tersebut, perpustakaan ingin membangun Sistem Informasi Manajemen Perpustakaan berbasis database.
2	Proses Bisnis Organisasi	Berikut adalah proses yang terjadi di perpustakaan: 1. Perpustakaan memiliki banyak buku. 2. Setiap buku memiliki informasi seperti: ○ Kode Buku ○ Judul Buku ○ Pengarang ○ Penerbit ○ Tahun Terbit ○ Kategori Buku ○ Jumlah Stok 3. Perpustakaan memiliki anggota yang dapat meminjam buku. 4. Setiap anggota memiliki data: ○ Id Anggota ○ Nama Anggota ○ Alamat ○ Nomor Telepon ○ Tanggal Daftar 5. Perpustakaan memiliki petugas yang melayani transaksi. 6. Anggota dapat melakukan peminjaman buku. 7. Dalam satu transaksi peminjaman: ○ Satu anggota dapat meminjam beberapa buku ○ Setiap transaksi dicatat oleh seorang petugas 8. Saat buku dikembalikan: ○ Sistem mencatat tanggal pengembalian ○ Jika terlambat, anggota dikenakan denda

3. Tugas Mahasiswa : Berdasarkan studi kasus di atas, lakukan beberapa langkah berikut.

1. Identifikasikan entitas utama dalam sistem perpustakaan. Tentukan juga atribut dari masing-masing entitas.

Entitas	Primary Key	Foreign Key	Atribut
Buku	Kode_Buku	-	○ Judul Buku ○ Pengarang ○ Penerbit ○ Tahun Terbit ○ Kategori Buku ○ Jumlah Stok
Anggota	Id_Anggota	-	○ Nama Anggota ○ Alamat ○ Nomor Telepon ○ Tanggal Daftar
Petugas	Id_Petugas	-	○ Nama Petugas ○ No Telp Petugas ○ Alamat Petugas
peminjaman	Id_Peminjaman	○ Id_Anggota ○ Id_Petugas	○ Tanggal peminjaman ○ Tanggal jatuh tempo ○ Tanggal Pengembalian ○ Total denda

2. Buatlah ERD sistem perpustakaan yang menggambarkan:

- Entitas
- Atribut
- Primary Key
- Relasi Antar Entitas
- Kardinalitas Relasi

Jawaban:

A. Identifikasi Entitas, Atribut, dan Primary Key

- **Entitas Buku**

Entitas Buku menyimpan data seluruh buku yang tersedia di perpustakaan. Setiap buku memiliki atribut Kode_Buku sebagai Primary Key. Atribut lainnya Adalah:

- Judul_Buku
- Pengarang
- Penerbit
- Tahun_Terbit
- Kategori_Buku
- Jumlah_Stok

- **Entitas Anggota**

Entitas Anggota menyimpan data seluruh anggota yang terdaftar pada perpustakaan dan memiliki hak untuk meminjam buku di perpustakaan tersebut. Setiap anggota memiliki atribut Id_Anggota sebagai Primary_Key. Atribut lainnya Adalah:

- Nama Anggota
- Alamat Anggota
- Nomor Telepon
- Tanggal Daftar

- **Entitas Petugas**

Entitas Petugas menyimpan data pegawai perpustakaan yang bertugas melayani dan mencatat transaksi peminjaman. Setiap Pegawai memiliki atribut Id_Petugas sebagai Primary Key. Atribut lainnya Adalah:

- Nama Petugas
- Nomor Telp Petugas
- Alamat Petugas

- **Entitas Peminjaman**

Entitas Peminjaman menyimpan data peminjaman yang terjadi di perpustakaan. Setiap peminjaman memiliki atribut Id_Peminjaman sebagai *Primary Key*. Dan juga entitas ini juga menyimpan Id_Anggota, Id_Petugas sebagai *Foreign Key*. Atribut lainnya Adalah:

- Tanggal peminjaman
- Tanggal jatuh tempo
- Tanggal Pengembalian
- Total Denda
- Tanggal Pengembalian

B. Relasi Antar Entitas dan Kardinalitas

1. Relasi "Melakukan" (Anggota ke Peminjaman)

One-to-Many(1:N)

Satu anggota (1) diperbolehkan untuk melakukan banyak (N) penyimpanan dari waktu ke waktu, sedangkan satu (1) transaksi peminjaman hanya dilakukan oleh satu anggota

2. Relasi "Melayani" (Petugas — Peminjaman)

One-to-Many(1:N)

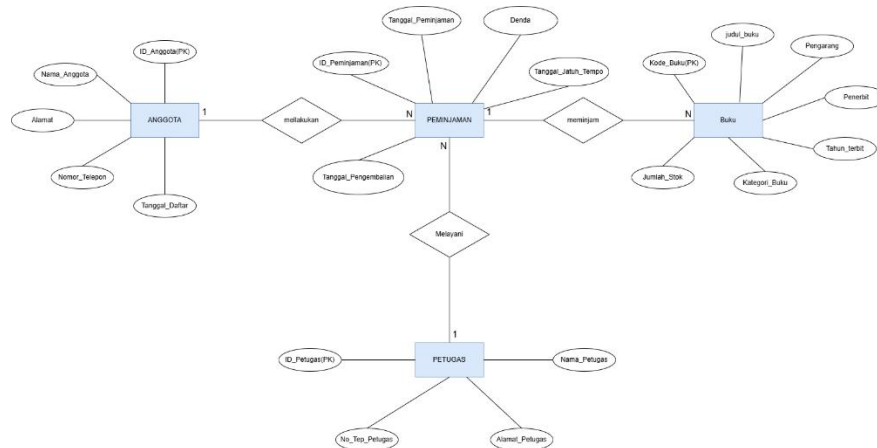
Satu petugas dapat mencatat banyak transaksi peminjaman, namun setiap transaksi hanya dicatat oleh satu petugas.

3. Relasi "Memiliki" (Buku ke Peminjaman)

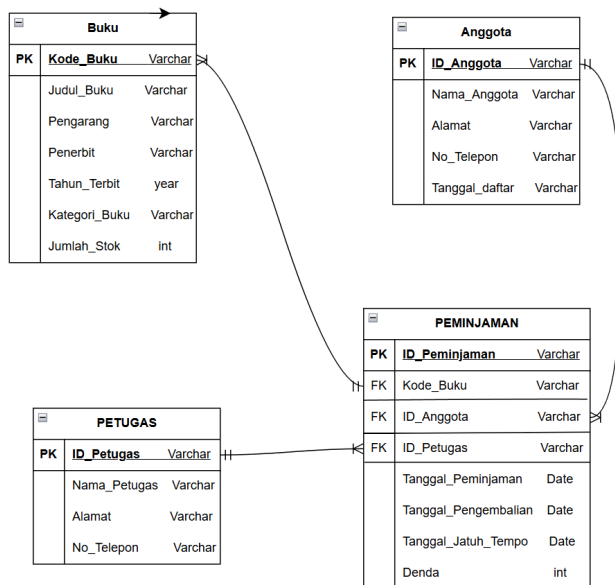
One-to-Many(1:N)

Satu buku dapat muncul di banyak riwayat transaksi peminjaman karena bisa dipinjam berkali-kali, namun setiap baris detail hanya merujuk pada satu buku.

C. Gambar ERD



3. Konversikan ERD yang telah dibuat menjadi struktur tabel relasional.



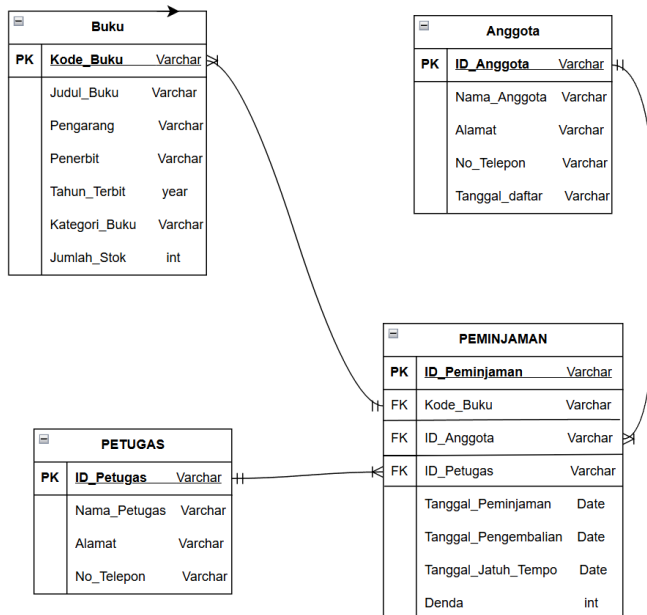
4. Lakukan normalisasi hingga Third Normal Form (3NF) jika terdapat redundansi data. Jelaskan proses normalisasi yang dilakukan.

UNF: UNF adalah kondisi awal data dimana semua informasi digabung dalam satu tabel besar tanpa aturan tertentu. Data sangat redundan, banyak pengulangan nilai, dan belum ada primary key yang jelas. Harus diubah ke 1NF agar data menjadi terstruktur, atomik, dan memiliki identitas unik di setiap barisnya.

No.	Normalisasi	Entitas	Atribut
1	UNF	Rekap Perpustakaan	id_peminjaman id_anggota nama_anggota Alamat nomor_telepon tanggal_daftar kode_buku judul_buku pengarang penerbit tahun_terbit kategori_buku jumlah_stok tanggal_peminjaman tanggal_pengembalian jumlah_buku denda id_petugas nama_petugas

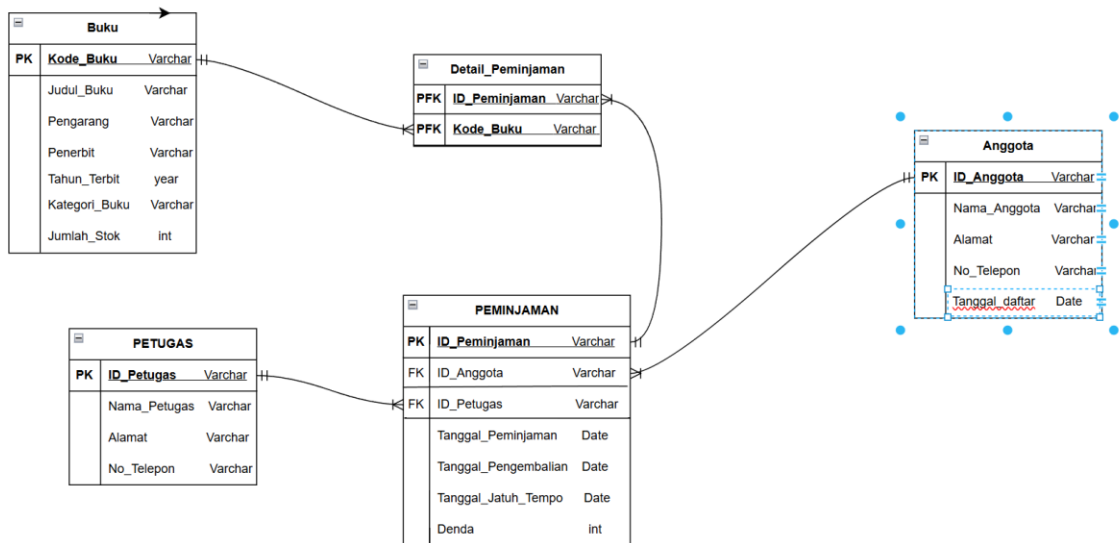
1NF

Proses Normalisasi yang dilakukan adalah pada tahap awal saya memisahkan entitas menjadi 4 tabel dan masing-masing memiliki primary key Tunggal.



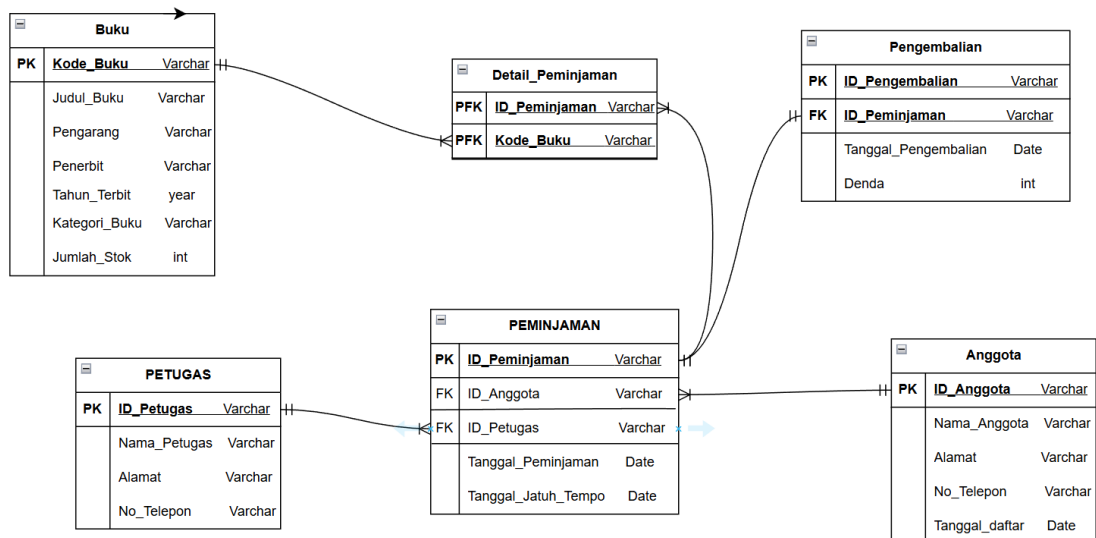
2NF

Selanjutnya saya menambahkan tabel detail_peminjaman yang menghubungkan table buku dengan table peminjaman untuk mengatasi relasi many-to-many (M:N)



3NF

Selanjutnya saya memisahkan lagi atribut pada table peminjaman menjadi table baru yaitu table pengembalian yang berisikan tanggal pengembalian dan denda. Tujuan nya untuk memudahkan pengembangan database dan strukturnya menjadi terorganisir. Contohnya Ketika menginginkan satu peminjaman mwmiliki tanggal jatuh tempo yang berbeda, sehingga table pengembalian ini sangat cocok



5. Tuliskan query SQL DDL untuk membuat tabel-tabel berikut: Buku, Anggota, Petugas, Peminjaman, Detail_Peminjaman, Pengembalian. Gunakan primary key dan foreign key.

1. Tabel Buku

```
CREATE TABLE Buku (  
Kode_Buku VARCHAR(10) PRIMARY KEY,  
Judul_Buku VARCHAR(100),  
Pengarang VARCHAR(100),  
Penerbit VARCHAR(100),  
Tahun_Terbit YEAR,  
Kategori_Buku VARCHAR(50),  
Jumlah_Stok INT  
);
```

2. Tabel Anggota:

```
CREATE TABLE Anggota (  
ID_Anggota VARCHAR(10) PRIMARY KEY,  
Nama_Anggota VARCHAR(100),  
Alamat VARCHAR(150),  
Nomor_Telepon VARCHAR(15),  
Tanggal_Daftar DATE  
);
```

3. Tabel Petugas

```
CREATE TABLE Petugas (  
ID_Petugas VARCHAR(10) PRIMARY KEY,  
Nama_Petugas VARCHAR(100),  
Alamat VARCHAR(150),  
Nomor_Telepon VARCHAR(15)  
);
```

4. Tabel Peminjaman

```
CREATE TABLE Peminjaman (  
ID_Peminjaman VARCHAR(10) PRIMARY KEY,  
ID_Anggota VARCHAR(10),  
ID_Petugas VARCHAR(10),  
Tanggal_Peminjaman DATE,  
Tanggal_Jatuh_Tempo DATE,
```

```
FOREIGN KEY (ID_Anggota) REFERENCES Anggota(ID_Anggota),  
FOREIGN KEY (ID_Petugas) REFERENCES Petugas(ID_Petugas)  
);
```

5. Tabel Detail_Peminjaman

```
CREATE TABLE Detail_Peminjaman (  
ID_Peminjaman VARCHAR(10),  
Kode_Buku VARCHAR(10),  
  
PRIMARY KEY (ID_Peminjaman, Kode_Buku), FOREIGN KEY (ID_Peminjaman)  
REFERENCES Peminjaman(ID_Peminjaman), FOREIGN KEY (Kode_Buku)  
REFERENCES Buku(Kode_Buku)  
);
```

6. Tabel Pengembalian

```
CREATE TABLE Pengembalian (  
ID_Pengembalian VARCHAR(10) PRIMARY KEY,  
ID_Peminjaman VARCHAR(10),  
Tanggal_Pengembalian DATE,  
Denda INT,  
  
FOREIGN KEY (ID_Peminjaman) REFERENCES Peminjaman(ID_Peminjaman)  
);
```

6. Tuliskan query SQL untuk: Menampilkan daftar buku yang tersedia, Menampilkan buku yang sedang dipinjam oleh anggota tertentu, Menampilkan jumlah buku yang dipinjam setiap anggota, Menampilkan anggota yang terlambat mengembalikan buku.

1. Menampilkan Daftar Buku Yang Tersedia

```
SELECT * FROM Buku  
WHERE Jumlah_Stok > 0;
```

2. Menampilkan buku yang sedang dipinjam oleh anggota tertentu

```
SELECT b.Judul_Buku, dp.Jumlah_Buku, p.Tanggal_Peminjaman  
FROM Peminjaman p  
JOIN Detail_Peminjaman dp ON p.ID_Peminjaman = dp.ID_Peminjaman  
JOIN Buku b ON dp.Kode_Buku = b.Kode_Buku  
LEFT JOIN Pengembalian k ON p.ID_Peminjaman = k.ID_Peminjaman  
WHERE p.ID_Anggota = 'A001'  
AND k.ID_Peminjaman IS NULL;
```


3. Menampilkan Jumlah Buku Yang dipinjam setiap anggota

```
SELECT a>Nama_Anggota, SUM(dp.Jumlah_Buku) AS Total_Buku_Dipinjam
FROM Anggota a
JOIN Peminjaman p ON a.ID_Anggota = p.ID_Anggota
JOIN Detail_Peminjaman dp ON p.ID_Peminjaman = dp.ID_Peminjaman
GROUP BY a.ID_Anggota, a>Nama_Anggota;
```

4. Menampilkan anggota yang terlambat mengembalikan buku

```
SELECT a>Nama_Anggota, p.ID_Peminjaman,
p.Tanggal_Jatuh_Tempo,
k.Tanggal_Pengembalian,
(DATEDIFF(k.Tanggal_Pengembalian,
p.Tanggal_Jatuh_Tempo)) AS
Hari_Terlambat FROM Pengembalian k
JOIN Peminjaman p ON k.ID_Peminjaman = p.ID_Peminjaman
JOIN Anggota a ON p.ID_Anggota = a.ID_Anggota
WHERE k.Tanggal_Pengembalian > p.Tanggal_Jatuh_Tempo;
```

7. Jelaskan bagaimana cara melakukan: Pengujian Integritas Data, Pengujian Transaksi Peminjaman, dan Optimasi Query Database

1. Pengujian Integritas Data

Pengujian integritas data bertujuan untuk memastikan bahwa data yang disimpan dalam database merupakan data yang valid, konsisten, dan sesuai dengan batasan yang telah dirancang dalam sistem informasi perpustakaan.

- **Entity Integrity (Integritas Entitas)**

Memastikan setiap baris data dalam tabel memiliki identitas yang unik melalui Primary Key. Primary Key tidak boleh bernilai NULL dan tidak boleh duplikat. Pengujian dilakukan dengan mencoba memasukkan data dengan Id_Anggota atau Kode_Buku yang sudah ada di dalam sistem. Sistem harus menolak eksekusi tersebut secara otomatis untuk mencegah terjadinya duplikasi data.

- **Referential Integrity (Integritas Referensial)**

Memastikan bahwa nilai Foreign Key pada suatu tabel selalu merujuk pada data yang benar benar ada di tabel induknya. Pengujian dilakukan dengan mencoba memasukkan transaksi Peminjaman menggunakan Id_Anggota yang tidak terdaftar di tabel Anggota. Database harus mengeluarkan error secara otomatis karena melanggar aturan Foreign Key. Selain itu pengujian juga dilakukan dengan mencoba menghapus data Anggota yang masih memiliki riwayat peminjaman aktif, sistem harus menolaknya atau melakukan Cascade sesuai rancangan.

- **Domain Integrity (Integritas Domain)**

Memastikan bahwa nilai yang dimasukkan sesuai dengan tipe data, format, dan batasan yang telah ditentukan untuk setiap kolom. Misalnya mencoba mengisi kolom Jumlah_Stok dengan angka negatif, atau mengisi Tanggal_Peminjaman dengan format huruf. Sistem harus memblokir input yang tidak sesuai dengan tipe datanya.

2. Cara Melakukan Pengujian Transaksi Peminjaman

Tabel sifat ACID dalam konteks perpustakaan, plus dua *stored procedure* lengkap: proses_peminjaman (dengan pengecekan stok, FOR UPDATE locking, dan ROLLBACK otomatis jika error) dan proses_pengembalian. Cara mengujinya :

- **Menggunakan Blok Transaksi (Database Transaction):**
Membungkus serangkaian query dengan perintah BEGIN TRANSACTION, COMMIT, dan ROLLBACK.
- **Skenario Sukses:** Melakukan INSERT ke tabel Peminjaman, lalu INSERT ke tabel Detail_Peminjaman, dan dilanjutkan UPDATE mengurangi Jumlah_Stok di tabel Buku. Jika ketiga proses ini berhasil tanpa error, maka jalankan perintah COMMIT untuk menyimpan perubahan secara permanen.
- **Skenario Gagal (Rollback):** Menguji apa yang terjadi jika di tengah proses terjadi kegagalan (misalnya, saat proses UPDATE stok gagal karena stok buku ternyata 0). Sistem harus memicu perintah ROLLBACK agar data Peminjaman dan Detail_Peminjaman yang baru saja di-insert ikut dibatalkan/dihapus secara otomatis, sehingga tidak ada data yang tersimpan setengah-setengah.

3. Cara Melakukan Optimasi Query Database

Optimasi query database bertujuan untuk meningkatkan kecepatan dan efisiensi proses pengolahan data, terutama ketika jumlah data yang tersimpan semakin besar seiring berjalannya sistem perpustakaan.

- **Penerapan Indexing**
Membuat Index pada kolom-kolom yang paling sering digunakan dalam klausa WHERE, JOIN, atau ORDER BY seperti kolom Id_Anggota, Kode_Buku, dan Id_Peminjaman. Dengan adanya index, pencarian data menjadi jauh lebih cepat.
- **Menghindari SELECT ***
mengambil semua kolom meski tidak dibutuhkan. Selalu tulis kolom yang diperlukan saja, misalnya SELECT KodeBuku, JudulBuku. Lebih ringan dan lebih cepat.
- **Menggunakan EXPLAIN**
Menambahkan perintah EXPLAIN sebelum query SELECT yang kompleks. Database Management System (DBMS) akan menampilkan rencana eksekusi (Execution Plan) sehingga pengembang bisa melihat di mana letak bottleneck (kemacetan) dari query tersebut dan bagian mana yang perlu ditambahkan index.